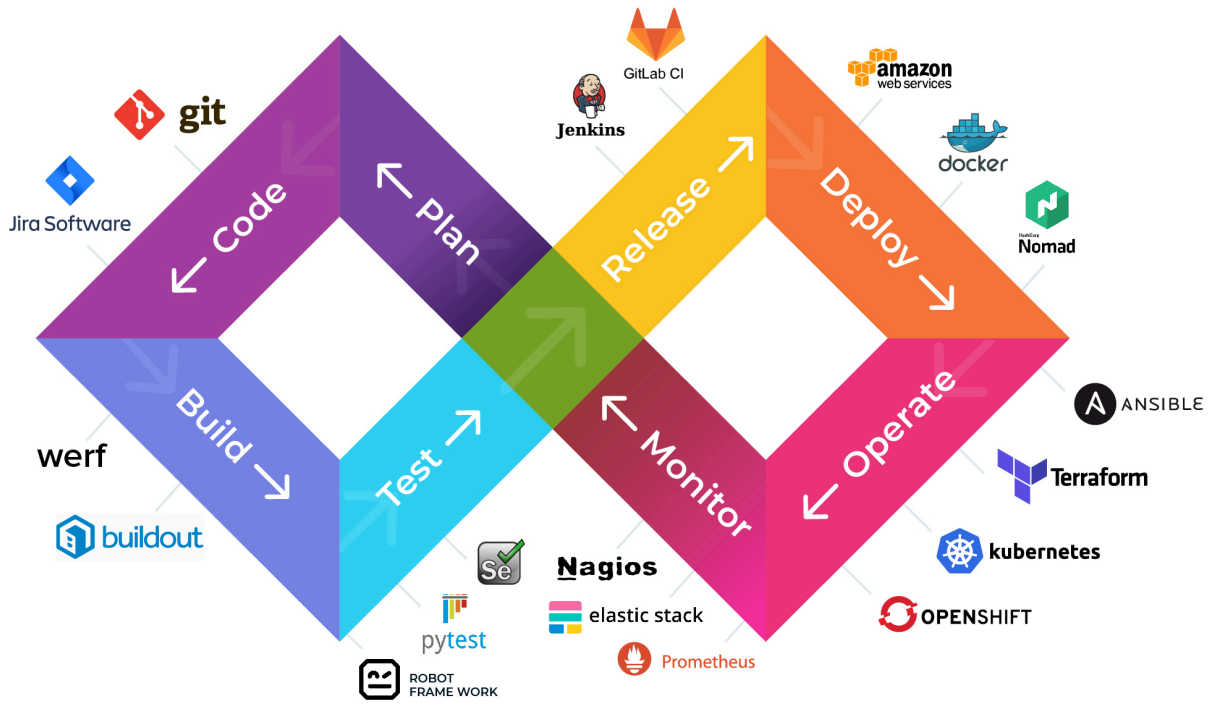
Docker

https://tinyurl.com/wsi-docker-swarm



Main example: https://testdriven.io/blog/running-flask-on-docker-swarm/

Installation

https://www.digitalocean.com/community/tutorials/how-to-install-and-use-docker-on-debian-10

```
docker swarm init --advertise-addr 10.20.0.1  # current node becomes manager, for swarm
```

Services

https://docs.docker.com/compose/compose-file/

```
version: "3"
services:
  myservice:
    image: wsiz/aio:1.0.5
    deploy:
      replicas: 2
      resources:
        limits:
          cpus: "0.2"
          memory: 50M    # good idea; else whole host memory is av. to container



       restart_policy:
       condition: on-failure

       ports:
       - "4000:8080"

       networks:
       - webnet

networks:
  webnet:
```

Run in swarm

```
docker stack deploy -c docker-compose.yaml swarm1
```

Main command table

```
docker build -t wsiz/aio:1.0.3 .

docker service ls
docker service ps mswarm1_web        #see where copies are located
docker service rm mswarm1_web
docker service logs mswarm1_web
docker service logs --follow mswarm1_web # also logs during deployment
docker service scale mswarm1_web=6   #service must be published to dockerhub

docker service inspect --pretty mswarm1_web  # full info ~yaml file

docker service update --image wsiz/aio:1.0.2 mswarm1_aaa (?)


docker node update --availability drain boston             #all removed from host `boston`
docker node ls                                             #status of nodes (also -q)
docker node promote <node name>                            #promote to extra manager
docker node update --label-add <key>=<value> <node-id>
docker node update --label-add type=primary <node-id>
docker node inspect --pretty <node-id>                     #shows IPs, labels, keys
```

```
# stack == set of services

docker stack services mswarm1
docker stack ps mswarm1                                 #lists services and nodes
docker stack deploy -c docker-compose.yml mswarm1       #also updates the service
docker service logs --follow mswarm1_cdn        # also --tail
```

```
docker login                                #log in to dockerhub
docker push username/image_name:tag         #publish to dockerhub
```

```
docker swarm init --advertise-addr 10.10.0.4   # create swarm; chose IP
docker swarm join-token worker                 # get the proper token to join the swarm
docker swarm join --token <token>              # execute from new docker node
```

Node Labels

https://docs.docker.com/datacenter/ucp/2.2/guides/admin/configure/add-labels-to-cluster-nodes/

```
docker node update --label-add hostn=boston boston
```

labels can be used in deploy specifications:

```
deploy:
   placement:
     constraints:
       - node.labels.disk == ssd
       - node.labels.hostn == madrid
```

searching nodes:
`docker node ls`

with labels:
`docker node ls -f node.label=hostn=cdn`

```
(base) pm@boston:~/deploy/cdn$ docker node ls -f node.label=hostn=cdn
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
u96npz6dbmp36420cw5ki5xl6     swarmasia  Down      Drain                           19.03.7
```

Operations

reboot of a node:

- `docker node update --availability drain madrid`

When you set the node back to Active availability, it can receive new tasks:

- during a service update to scale up
- during a rolling update
- when you set another node to Drain availability
- when a task fails on another active node

Docker secrets

(properties of a swarm; stored encrypted)

`create/store`

```
echo "aaa#1234" | docker secret create pm_pwd -
```

If container has access to `pm_pwd`, it will be visible under file:
`/run/secrets/pm_pwd`

`adding/removing access to a service: (secret-add/-rm)`

docker service update --secret-add pm_pwd mswarm1_aio

In docker-compose.yml:

```
        secrets:
          - pm_pwd

secrets:
  pm_pwd:
        file: pm_pwd.prod.txt
  other:
      external: true
```

Note: cannot "update" secrets; must remove, and then deploy the service again,
`docker secret rm mswarm1_auth_db_pass`

Rolling update

https://docs.docker.com/engine/swarm/swarm-tutorial/rolling-update/

Health

https://docs.docker.com/engine/reference/builder/#healthcheck

Health:
https://blog.couchbase.com/docker-health-check-keeping-containers-healthy/
https://howchoo.com/g/zwjhogrkywe/how-to-add-a-health-check-to-your-docker-container

--but should not use curl, as most images do not have it:
https://blog.sixeyed.com/docker-healthchecks-why-not-to-use-curl-or-iwr/

```
      networks:
        - webnet
      healthcheck:
        test: "python check.py"
        start_period: 5s
        interval: 20s
        timeout: 1.2s
        retries: 5

networks:
  webnet:
```

- timeout should be sufficient, 1s is not sufficient (and restart loop ensues)
- waits 5s, then 20s, then reports "running", but service available earlier

https://docs.docker.com/compose/compose-file/

Images

docker image prune --all #removes unused images

Dockerhub private repos

https://devops.stackexchange.com/questions/9444/access-private-dockerhub-repo-in-docker-swarm?newreg=fc1362b8e87f41789e741bac7f131296

```
docker stack deploy -c docker-compose.yml --with-registry-auth mswarm1
```

Monitoring

```
docker stats
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
77e31b592087	brave_spence	0.06%	134.7MiB / 150MiB	89.79%	219kB / 105kB	63MB / 0B	31
9fbcd976b1ef	cloud_prometheus.1.z9z9kbxzc0n47wvxtpt6x4cz6	0.00%	70.35MiB / 31.33GiB	0.22%	41.9kB / 0B	65.8MB / 34.1MB	30
6374a0189465	cloud_grafana.1.9j4yqyubl9auomkr1fj1sqxbi	0.01%	38.61MiB / 31.33GiB	0.12%	7.01MB / 2.53MB	43.2MB / 12.3MB	30
266c427ea034	cloud_xdb.1.hxfjao0obxr1mtw6nkxckigap	0.00%	40.33MiB / 31.33GiB	0.13%	41.7kB / 0B	30.4MB / 187MB	7

docker system df

```
root@swarmeurope:~# docker system df
TYPE                TOTAL               ACTIVE              SIZE                RECLAIMABLE
Images              105                 14                  4.632GB             4.06GB (87%)
Containers          15                  15                  2.983MB             0B (0%)
Local Volumes       2                   1                   98.72MB             98.72MB (100%)
Build Cache         0                   0                   0B                  0B
```

Logging/troubleshooting

```
docker service logs mswarm1_replayapi
```

→

```
mswarm1_replayapi.2.45qfmvy3hzt8@boston | Error handling request
mswarm1_replayapi.2.45qfmvy3hzt8@boston | Traceback (most recent call last):
mswarm1_replayapi.2.45qfmvy3hzt8@boston |   File
"/usr/local/lib/python3.8/site-packages/aiohttp/web_protocol.py", line 418, in start
mswarm1_replayapi.2.45qfmvy3hzt8@boston |      resp = await task
```

(instance, host, error)

```
docker events              # displays all events live…
```

Disaster recovery

https://docs.docker.com/ee/admin/disaster-recovery/

Docker networking

https://neuvector.com/network-security/docker-swarm-container-networking/

overlay -- swarm-scope network accessible to all services (deployed with it as network)

```
             node 1

 172.18.0.4         172.18.0.3
+----+------------------+----------------+ docker_gwbridge
     |                  |
     |                  |
     |                  |
     |                  |
  +--+--------+      +--+--------+
  |  nodejs   |      |   redis   |
  |           |      |           |
  +--------+--+      +--------+--+
           |                  |
           |                  |
           |                  |
           |                  |
      +----+------------------+----------+ net1
       101.0.0.3          101.0.0.8
       101.0.0.4(vip)     101.0.0.2(vip)
```

defined network (custom): net1 (overlay)
the (host-local) docker_gwbridge allows to access containers from (node 1) host

- For Docker Swarm services that publishes ports (with the -p option), Docker creates a dedicated ‘ingress’ network
- One of the major features of Docker Swarm is the ‘routing mesh’ for containers that publish ports. No matter which node the container instance is actually running on, you can access it through any node. How is it done?

nginx proxy

It makes sense to have a (https) nginx proxy, forwarding to all nodes of the swarm
config:

```
upstream swarm1 {
  server 10.10.0.200:8080;
  server 10.10.0.107:8080;
  server 10.10.0.30:8080;
}


server {
  listen 443 ssl;
  server_name sw.wsi.edu.pl www.sw.wsi.edu.pl;
  ssl_certificate wsi_full.crt;
  ssl_certificate_key wsi.key;
  ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
  ssl_ciphers HIGH:!aNULL:!MD5;

  location / {
          #proxy_pass http://10.10.0.200:8080;
          proxy_pass http://swarm1;
          #proxy_set_header Host $host;
          #proxy_set_header X-Real-IP $remote_addr;
  }
}
```

note -- will get "bad gateway 502" due to selinux, due to proxying http to 8080, not defined for nginx; install semanage
https://www.nginx.com/blog/using-nginx-plus-with-selinux/

```
(yum provides /usr/sbin/semanage, yum install policycoreutils-python-2.5-33.el7.x86_64),
```

then:

```
grep nginx /var/log/audit/audit.log | audit2allow -M nginx
semodule -i nginx.pp
semodule -l | grep nginx
```

finally:
https://sw.wsi.edu.pl/